

SmartFetch: Final Project Report

Adaptive Web Data Extraction Web Application

Prepared by

Youssef Mabrouk

Adam Hajri

Jaouher Chouchane

Mohammed Sanad

Course

Information Assurance and Security

Instructor

Prof. Manel Abdelkader

Date

May 2026

Contents

1	Introduction	1
2	Project Objective	1
3	Scope	2
4	Web Scraping Concepts	3
5	Final Application Overview	3
6	Technology Stack	4
7	High-Level Architecture	4
8	System Flow	6
9	API Design	6
10	Adaptive Filters	7
11	Separate and Grouped Extraction Modes	8
12	Data Cleaning	9
13	Export Design	9
14	Security and Ethical Considerations	10
15	Testing Strategy	10
16	Discussion	11
17	Limitations	12
18	Future Improvements	12
19	Conclusion	13
A	User Interface Screenshots	14
B	Sample Extraction Output	15
	References	17

Abstract

SmartFetch is a client-server web application designed for adaptive web data extraction from a single user-provided webpage. Unlike traditional fixed scrapers, the system dynamically analyzes the HTML structure of the page and generates context-aware extraction filters. The application supports both separate and grouped extraction modes, allowing users to either extract individual data types or preserve relationships between related elements such as headings, paragraphs, links, images, prices, and dates. The backend uses Node.js, Express.js, Axios, and Cheerio to retrieve webpages, parse HTML, extract selected data, clean the output, and prepare exports. The system also integrates basic security mechanisms such as URL validation, protocol restriction, private IP blocking, request timeout, and response size limitation. This report presents the final SmartFetch application, including its objectives, scope, architecture, system flow, implementation choices, testing strategy, limitations, and future improvements.

1 Introduction

Manual web data extraction is time-consuming, repetitive, and prone to errors. Webpages often contain useful information, but this information is usually embedded inside HTML structures that vary from one website to another. As a result, a scraper designed for one page structure may fail when the layout changes or when another website uses different tags and containers. Web scraping is commonly used to collect online data in a structured format, but effective scraping requires careful parsing, cleaning, and ethical use of publicly available information (Mitchell, 2018).

SmartFetch addresses this problem by introducing an adaptive approach to web scraping. Instead of relying on a fixed set of extraction rules, the system first analyzes the structure of the webpage and then suggests filters based on the detected content. This makes the extraction process more flexible and user-friendly.

The project is intentionally limited to a single webpage supplied by the user. SmartFetch is not a full crawler and does not automatically follow links across an entire website. This design choice keeps the system easier to control, safer to test, and more realistic for the scope of the project.

2 Project Objective

The main objective of SmartFetch is to provide a controlled and adaptive workflow for extracting useful data from webpages. The application allows users to analyze a page, select relevant filters, preview results, and export structured output.

More specifically, SmartFetch aims to:

1. Accept one public webpage URL from the user.
2. Validate the URL and retrieve the HTML content.
3. Analyze the HTML structure to detect available data types.

4. Suggest extraction filters dynamically.
5. Extract only the data selected by the user.
6. Clean the extracted output by removing empty values, trimming whitespace, and eliminating duplicates.
7. Preview the results in the browser.
8. Export the final result as JSON or CSV.

The core goal is not to scrape every possible element from a webpage, but to help users choose which data is meaningful and extract it in a clean, reusable format.

3 Scope

SmartFetch focuses on adaptive extraction from one webpage at a time. The current version supports static HTML retrieval and backend parsing. It does not execute JavaScript on target websites, does not handle authenticated pages, and does not crawl multiple pages.

Table 1: SmartFetch Scope Boundaries

Included in Scope	Outside Current Scope
Single public webpage analysis	Full-site crawling
HTML parsing with Cheerio	JavaScript-rendered page execution
Adaptive filter suggestions	Login or authenticated scraping
Preview and export as JSON/CSV	Scheduled or background scraping
Basic URL and SSRF protection	Proxy rotation or anti-bot bypassing
Separate and grouped extraction modes	Large-scale crawling infrastructure

This limited scope supports the project’s security and feasibility goals. It also makes the system easier to demonstrate and test within an academic environment.

4 Web Scraping Concepts

A web scraper generally sends an HTTP request to a target website, receives the page content as HTML, parses the HTML structure, extracts selected data, cleans the result, and stores or exports it in a structured format. SmartFetch follows this general process but avoids crawler behavior.

The main concepts used in SmartFetch are:

- **Target Website:** the public webpage selected by the user.
- **Request and Response:** the backend sends an HTTP request and receives HTML content.
- **Parser:** Cheerio is used to inspect and query HTML elements.
- **Data Extraction:** selected filters are applied to extract useful information.
- **Data Cleaning:** empty values, duplicates, and formatting issues are removed.
- **Export:** the final output is made available as JSON or CSV.

5 Final Application Overview

The final SmartFetch application provides a web interface organized around four main user actions:

- **URL Input:** the user enters a public webpage URL.
- **Suggested Filters:** the application displays filters based on detected page elements.
- **Preview:** extracted data is displayed in a table or card-style view.
- **Export:** the user downloads the result as JSON or CSV.

The interface also includes grouped extraction mode, which allows related content to be grouped together when it appears inside the same section, article, product card, or repeated block.

6 Technology Stack

The system uses a simple and realistic web development stack suitable for a university project.

Table 2: Main Implementation Technologies

Technology	Role in SmartFetch
HTML	Defines the frontend page structure.
CSS	Provides the visual design, responsive layout, panels, buttons, and preview styling.
JavaScript	Handles frontend interaction, API calls, filter selection, preview rendering, recent URL history, and downloads.
Node.js	Runs the backend server.
Express.js	Defines API routes and serves the static frontend.
Axios	Retrieves HTML from target webpages with timeout and redirect handling.
Cheerio	Parses HTML on the backend and extracts elements using CSS selectors.
JSON	Used for API communication and structured export.
CSV	Used for spreadsheet-friendly export.

7 High-Level Architecture

SmartFetch follows a client-server architecture. The frontend is responsible for collecting user input, displaying suggested filters, showing preview results, and triggering exports. The backend is responsible for URL validation, webpage retrieval, HTML parsing, filter generation, data extraction, cleaning, and response preparation.

This separation of responsibilities makes the system easier to maintain. The frontend remains focused on user interaction, while the backend centralizes the technical scraping logic. The architecture also improves security because validation and request handling are performed server-side.

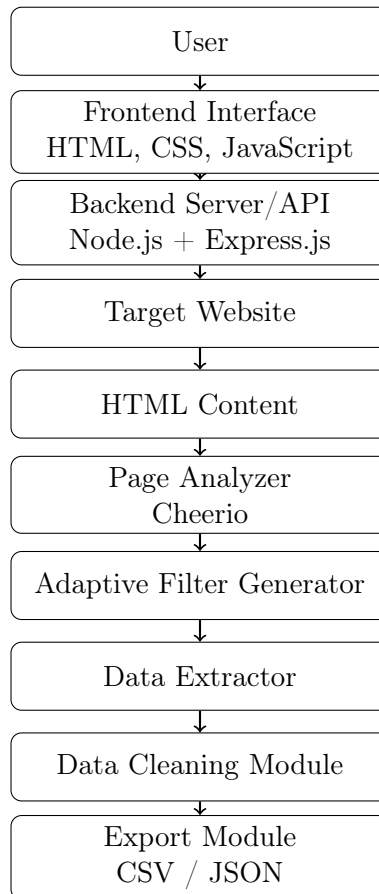


Figure 1: SmartFetch High-Level Architecture

The arrows represent the flow of data through the system. The user submits a URL through the frontend, which sends the request to the backend. The backend retrieves the webpage, processes the HTML content, generates adaptive filters, extracts and cleans the data, and finally prepares the output for export.

8 System Flow

The SmartFetch workflow is divided into two main phases: analysis and extraction. During the analysis phase, the system retrieves the webpage and detects available filters. During the extraction phase, the user selects filters, and the backend extracts, cleans, and returns the requested data.

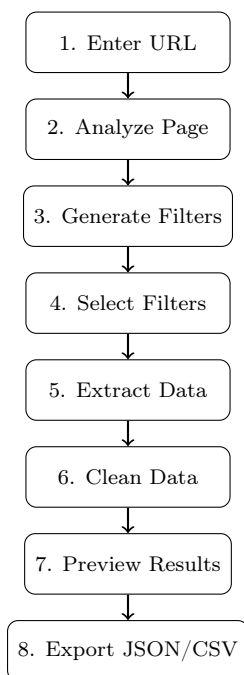


Figure 2: End-to-End SmartFetch Workflow

Figure 2 summarizes the user-facing workflow from URL submission to export. The system flow ensures that users do not need to define technical selectors manually. Instead, SmartFetch analyzes the webpage first and provides filter options based on detected content.

9 API Design

SmartFetch uses two main backend endpoints.

9.1 Analyze Endpoint

The `POST /api/analyze` endpoint receives a URL, validates it, retrieves the webpage, parses the HTML, and returns detected filters. The frontend uses this response to display selectable filter options.

9.2 Extract Endpoint

The `POST /api/extract` endpoint receives a URL, selected filters, and the grouped extraction option. It extracts the selected data, cleans the result, computes a summary, and returns structured output with CSV data.

Table 3: SmartFetch API Endpoints

Endpoint	Input	Output
<code>/api/analyze</code>	URL	Final URL, page title, suggested filters, counts
<code>/api/extract</code>	URL, filters, grouped mode	Extracted sections, summary, export settings, CSV data

Separating analysis from extraction improves usability. The system first understands the webpage structure before asking the user what to extract.

10 Adaptive Filters

SmartFetch detects filters dynamically based on the target HTML. This differs from fixed scrapers because the available extraction options change depending on the page.

Table 4: Detected SmartFetch Filters

Filter	Detection Source	Extracted Data
Headings	<code>h1, h2, h3</code>	Heading level and text
Paragraphs	<code>p</code> tags	Readable text content
Links	<code>a[href]</code>	Link text and resolved URL
Images	<code>img</code>	Image source and alt text
Tables	<code>table, tr, th, td</code>	Row values and structured columns
Lists	<code>ul, ol, li</code>	List items grouped by list type
Metadata	<code>title, meta</code> tags	Title, description, keywords, and Open Graph metadata
Prices	Price-like text patterns	Currency values such as \$, EUR, TND, and numeric price formats
Dates	<code>time, datetime,</code> date-like text	Date values from semantic tags and page text

The adaptive filter system makes the application flexible across multiple page types, including articles, product pages, table-based pages, and general websites.

11 Separate and Grouped Extraction Modes

SmartFetch supports two extraction modes.

- **Separate Mode:** each selected filter is extracted independently. This is useful when the user wants a simple list of headings, links, tables, or images.
- **Grouped Mode:** related elements are grouped when they appear inside the same repeated block, card, article, product section, or search result.

Grouped extraction is one of the strongest features of SmartFetch. It preserves contextual relationships between elements. For example, a product name, price, image, and link that appear inside the same card are grouped into one structured record instead of

being extracted as unrelated lists. This improves the usability of exported data because the output reflects how content is organized on the webpage.

Grouped mode is especially useful for product listings, news cards, article previews, search results, and repeated content blocks. However, it relies on heuristics, so very unusual page structures may still produce imperfect grouping.

12 Data Cleaning

Raw web data often contains empty strings, duplicate values, navigation links, footer links, extra spaces, and repeated elements. SmartFetch applies cleaning before preview and export.

The cleaning process includes:

- trimming whitespace and normalizing repeated spaces;
- removing empty rows and empty values;
- removing duplicate rows;
- resolving relative URLs into absolute URLs;
- reducing noisy navigation, footer, account, cookie, and repeated site-wide links;
- deduplicating grouped links by destination URL;
- keeping image previews as thumbnails while exporting image URLs.

These steps make the extracted data easier to read, preview, and reuse.

13 Export Design

SmartFetch supports two main export formats.

- **JSON export:** preserves the complete response structure, including source URL, selected filters, grouped mode, extracted sections, and extracted rows.

- **CSV export:** creates spreadsheet-friendly rows that can be opened in applications such as Microsoft Excel or Google Sheets.

For image extraction, the export stores image URLs rather than embedding binary image data. For table extraction, the backend attempts to preserve meaningful columns whenever possible.

14 Security and Ethical Considerations

SmartFetch is designed for public webpage data only. It should not be used to collect private, sensitive, or personally identifiable information. Since web scraping can create technical and ethical risks, the application includes several basic controls.

- invalid URLs are rejected;
- only `http` and `https` protocols are allowed;
- localhost and private IP ranges are blocked where possible;
- redirects are checked before being followed;
- requests are limited with a timeout;
- HTML response size is limited;
- the system processes one URL at a time instead of crawling many pages.

These controls reduce risk but do not replace responsible use. Users should respect website terms of service, robots policies, privacy rules, and reasonable request limits.

15 Testing Strategy

The system was tested using backend tests and functional validation. The testing focused on extraction correctness, cleaning behavior, grouped extraction, export consistency, and security checks.

The tests covered:

- whitespace normalization and duplicate removal;
- extraction of headings, links, images, tables, prices, dates, lists, and metadata;
- grouped extraction behavior;
- duplicate link removal in grouped mode;
- table-like layout extraction;
- CSV export with settings;
- URL validation and private IP blocking.

The backend test suite executed successfully during development, confirming that the main extraction and cleaning functions behave as expected. Functional testing was also performed using different types of webpages, including news-style pages, product pages, and table-based pages.

16 Discussion

SmartFetch demonstrates that adaptive extraction can improve flexibility compared to fixed scraping approaches. By detecting available page elements first, the system avoids forcing the same filters on every webpage. The grouped extraction mode adds further value by preserving context between related elements, which is important when scraping product cards, articles, or repeated search results.

However, the current implementation also highlights common limitations of lightweight scraping systems. Since SmartFetch does not execute JavaScript, it may not capture content loaded dynamically after the initial page response. In addition, grouped extraction depends on heuristic detection of repeated blocks, so it may not always perfectly match the intended structure of complex pages.

Overall, the system balances functionality, simplicity, and security. It is suitable for academic demonstration while still reflecting realistic design choices used in web extraction tools.

17 Limitations

SmartFetch is functional and ready for demonstration, but it has realistic limitations:

- it does not execute JavaScript on target websites;
- it does not support login-protected pages;
- grouped extraction may be imperfect on unusual layouts;
- it does not include custom user-defined CSS selectors;
- it is intentionally not a full crawler.

18 Future Improvements

Possible future improvements include:

- optional browser automation for JavaScript-heavy websites;
- saved custom filter presets;
- more advanced repeated-block detection;
- controlled multi-page crawling with strict limits;
- richer export templates for specific use cases such as products, news, or research data;
- an in-app explanation panel for each filter and for grouped extraction mode.

19 Conclusion

SmartFetch successfully implements an adaptive approach to web data extraction. It combines a clean frontend interface with a backend pipeline for URL validation, HTML retrieval, page analysis, filter generation, extraction, cleaning, and export.

The adaptive filter approach makes the system more flexible than fixed scrapers, while grouped extraction improves the usefulness of the data by preserving relationships between related elements. The single-page scope and security controls keep the project realistic, controlled, and suitable for academic demonstration.

Future work could extend SmartFetch with browser automation, custom filter presets, and controlled multi-page crawling. However, the current version already fulfills the main objectives of the project by providing a working, structured, and adaptive web scraping application.

A User Interface Screenshots

This appendix presents screenshots of the final SmartFetch web application interface.

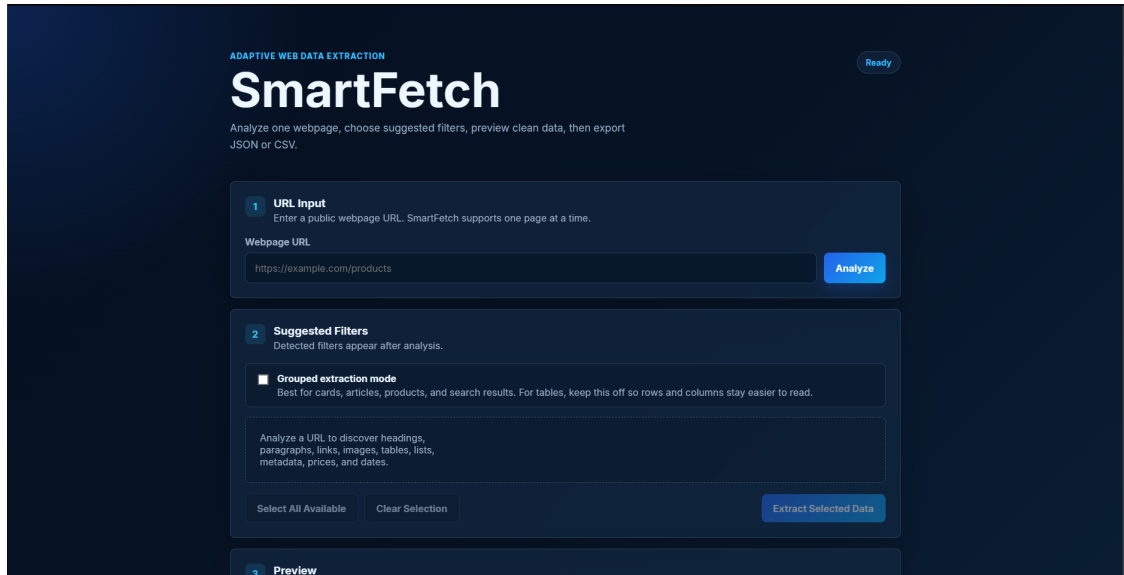


Figure 3: SmartFetch main interface showing the URL input and available filters.

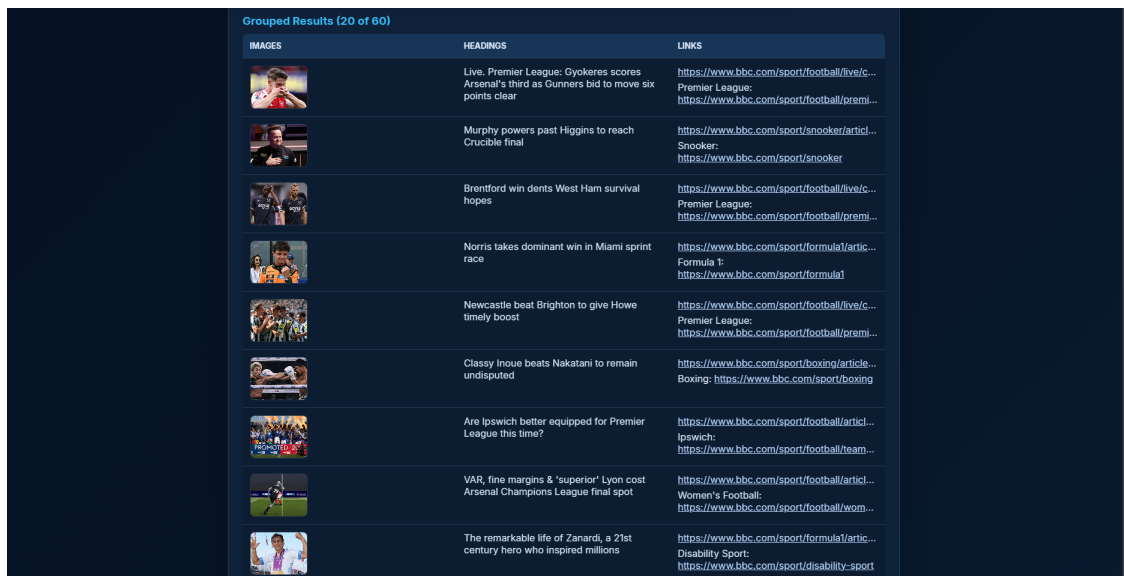


Figure 4: SmartFetch results preview after extracting data from a webpage.

B Sample Extraction Output

The following sample output was generated from the webpage <https://www.bbc.com/sport>. The extraction was performed in grouped mode using the filters Headings, Paragraphs, Links, Images, and Lists.

Table 5: Sample Export Summary

Setting	Value
Source URL	https://www.bbc.com/sport
Grouped Mode	true
Selected Filters	Headings, Paragraphs, Links, Images, Lists
Total Items	60
Extraction Duration	118 ms

B.1 JSON Output Sample

```
{
  "url": "https://www.bbc.com/sport",
  "grouped": true,
  "summary": {
    "totalItems": 60,
    "selectedFilters": [
      "Headings",
      "Paragraphs",
      "Links",
      "Images",
      "Lists"
    ],
    "filterCount": 5,
    "durationMs": 118
  },
  "sections": [
    {
      "key": "grouped",
      "label": "Grouped Results",
      "items": [
```

```

{
  "group": "1",
  "selector": "[data-testid*=\"promo\"]",
  "headings": "Live. Premier League: Gyokeres scores Arsenal's third...",
  "paragraphs": "Arsenal host Fulham after West Ham beaten by Brentford...",
  "links": "https://www.bbc.com/sport/football/live/c20qq1vjgv0t",
  "images": "Viktor Gyokeres scores for Arsenal: image URL",
  "lists": "AttributionPremier League"
}
]
}
]
}

```

B.2 CSV Output Sample

Table 6: Sample CSV-Style Output

Group	Heading	Paragraph	List / Category	
1	Live. Premier League: Gyokeres scores Arsenal's third as Gunners bid to move six points clear	Arsenal host Fulham after West Ham beaten by Brentford and Newcastle beat Brighton.	Attribution League	Premier
2	Murphy powers past Higgins to reach Crucible final	Murphy powers past Higgins to reach Crucible final	Attribution	Snooker
3	Brentford win dents West Ham survival hopes	Brentford win dents West Ham survival hopes	Attribution League	Premier

References

- Mitchell, R. (2018). *Web scraping with Python: Collecting more data from the modern web* (2nd ed.). O'Reilly Media.
- MDN Web Docs. (n.d.). *HTML: HyperText Markup Language*. Mozilla. <https://developer.mozilla.org/en-US/docs/Web/HTML>
- Node.js. (n.d.). *Node.js documentation*. <https://nodejs.org/en/docs>
- Express.js. (n.d.). *Express: Node.js web application framework*. <https://expressjs.com>
- Cheerio. (n.d.). *Cheerio documentation*. <https://cheerio.js.org>
- Axios. (n.d.). *Axios documentation*. <https://axios-http.com>